

PROJECT REPORT

AI Chat Bot Based Bus Ticketing System

A full-stack AI-powered travel assistant that finds, filters, and presents live bus tickets through natural language conversation.

3+ AI Models	10 Bus Scrapers	15+ Indian Cities	4 Smart Filters	8 UI Components
------------------------	---------------------------	-----------------------------	---------------------------	---------------------------

June 2026

1. What We Built

We built an **AI-powered bus ticket search system** that lets travellers find and compare real bus tickets by simply chatting in plain English. Instead of navigating complex booking websites, the user types something like *"Find me the cheapest AC bus from Madurai to Chennai tomorrow"* and the system instantly returns live, filtered, sorted results directly from RedBus.

Core Idea: Replace the traditional search form with a smart conversational AI that understands what the user wants, fetches live data, and shows only the most relevant results — no manual filtering required.

2. Problem We Solved

Bus booking in India has three major pain points:

- **Fragmented platforms** — RedBus, AbhiBus, MakeMyTrip all have different UIs and filters.
- **Manual filtering** — Users must manually sort by price, type, timing, operator.
- **Complex interfaces** — Non-technical travellers find booking portals confusing.
- **No single conversational interface** — Nobody accepts plain English queries.

3. What We Did — Feature by Feature

3.1 Conversational AI Chat Interface

Built a chat UI where users type natural language queries. The system extracts travel intent — source city, destination, date, bus type, price preference — using an AI model via OpenRouter. When the AI hits rate limits, a built-in rule-based parser handles the request as a fallback.

- Multi-turn conversation: bot asks follow-up questions if info is missing
- Quick-reply chips (Tomorrow / Day After / This Weekend) for faster input
- Chat history preserved within the session
- Voice input support using Web Speech API

3.2 Live Bus Data Scraping

Used Playwright (headless browser automation) to scrape live bus data from RedBus by intercepting its internal GraphQL API. Every search returns real, up-to-date bus information — not dummy or cached data.

Each bus result includes:

- Operator name (e.g. SETC, FlixBus, KPN, Orange Travels)
- Departure & arrival times, journey duration
- Price in INR (live from RedBus)

- Seats available (urgent flag if < 5 seats left)
- Bus type: AC / Sleeper / Volvo / Non-AC
- Amenities: WiFi, charging, live tracking
- Free cancellation flag
- Direct booking URL back to RedBus

3.3 Smart AI Filtering & Sorting

The AI detects filter and sort intent from the user's message and applies it server-side before returning results. The top result always matches what the user asked for.

Keyword Detected	What Happens
cheapest	Sorts all buses by price — lowest first. Badge: ■ CHEAPEST
ac	Filters to AC buses only. Badge: ❄️ ■ BEST AC BUS
sleeper	Filters to sleeper buses. Badge: ■ BEST SLEEPER
non_ac	Shows economy non-AC options only
volvo	Filters luxury/Volvo operators
fastest	Sorts by journey duration — shortest first
night bus	Shows buses departing after 8 PM only

3.4 Smart Connecting Route Suggestion

If no direct bus exists between two cities, the system automatically suggests a two-leg connecting route through a known intermediate stop. Examples: Chennai → Dindigul → Kodaikanal, Coimbatore → Mettupalayam → Ooty.

3.5 Premium Glassmorphism UI

Designed a 2026-style modern interface with glassmorphism panels, smooth animations, and a Dark / Light theme toggle that persists in localStorage.

- Dark/Light theme toggle — preference saved in localStorage
- Animated filter badges on the results panel
- "BEST AC BUS" / "CHEAPEST" / "BEST SLEEPER" highlight card on top result
- Micro-animations on hover for all interactive elements
- Backdrop-filter blur glass cards throughout the UI

3.6 AI Travel Guide Panel

After a bus search, a Travel Guide tab appears for the destination city. An AI model generates travel tips, top places to visit, local food recommendations, and best time to visit.

3.7 Interactive Map

Built using Leaflet.js — shows an interactive map with pin markers for tourist spots near the destination. Clicking a marker shows the place name and details.

3.8 Booking History Tracker

Users can complete a booking flow inside the app. The system generates a confirmation ID, stores the booking in localStorage, and shows a full booking history panel with operator, route, date, seat, and amount.

3.9 Bus Operator Reviews

Users can rate bus operators with a star rating. Ratings are displayed on each ticket card to help others pick the best operator.

3.10 User Personalisation

The app greets the user by name on return visits, remembers frequently searched routes, and suggests smart quick replies based on past searches.

4. Languages & Technologies Used

Frontend

React 18	Component-based UI framework for building the chat interface and results
TypeScript 5	Strongly-typed JavaScript — catches bugs at compile time
Vite 6	Build tool and dev server with Hot Module Replacement
Vanilla CSS	Full custom styling — glassmorphism, CSS variables, animations
Leaflet.js	Interactive map showing tourist spots near the destination
Lucide React	Clean icon library used throughout the UI
Web Speech API	Browser-native voice input for hands-free search
localStorage	Persists theme preference, booking history, chat sessions

Backend

Python 3.11+	Core programming language for the backend
FastAPI	Modern async REST API framework — fast, auto-generates docs
Uvicorn	ASGI server that runs the FastAPI application
Pydantic	Data validation and request/response schema definitions
httpx	Async HTTP client for making AI API calls to OpenRouter
python-dotenv	Loads API keys and config from .env files
Docker	Containerises the backend for easy deployment anywhere

AI & Scraping

OpenRouter API	Gateway to multiple free/paid LLM models
Nous Hermes 3 (Llama 3.1 405B)	Primary AI model for extracting travel intent from user messages
Rule-Based Parser	Regex fallback that handles queries when AI hits rate limits
Playwright	Async headless browser — intercepts RedBus GraphQL API for live data
BeautifulSoup	HTML parser used as secondary fallback for bus data
Git	Version control used throughout the project

5. How It All Works Together

The system follows a clean three-tier architecture. The flow from user message to final result takes under 10 seconds:

1	User sends a chat message React UI posts the query + conversation history to the FastAPI backend.
2	AI parses the intent OpenRouter calls the Nous Hermes 3 model which returns a structured JSON with from_city, to_city, date, filter, sort_by. If rate-limited, the rule-based regex parser takes over.

3	Bot asks follow-up if needed If any required field is missing, the bot sends a polite question with quick-reply chips to speed up input.
4	Playwright scrapes RedBus Once all fields are collected, a headless Chromium browser navigates to RedBus, intercepts the GraphQL API response, and returns live bus data.
5	Filter & Sort applied The filter_and_sort_buses() function applies the detected preference (e.g. sort by price for 'cheapest') before returning results.
6	Results shown in the UI React renders ticket cards with badges, booking links, and an active-filter banner at the top of the results panel.

6. Key Results & Achievements

Feature	Outcome
Live Bus Data	Real-time bus listings from RedBus — prices, seats, times, operators
AI Filtering	Correctly detects cheapest / AC / sleeper / Volvo / fastest / night
Filter Badges	Active filter banner + 'BEST AC BUS' badge shown on top card
Multi-turn Dialogue	Bot holds context across messages, asks only for what is missing
Connecting Routes	Auto-suggests 2-leg route when no direct bus found
Theme Toggle	Dark/Light mode switch — preference saved in localStorage
Travel Guide	AI generates destination tips after every bus search
Voice Search	Hands-free query via Web Speech API
Booking Tracker	Full in-app booking flow with confirmation ID and history
Zero AI Cost	Primary model on free tier; rule-based fallback means always works

7. What Can Be Improved Next

- **Payment Integration** — Add Razorpay / Stripe for end-to-end booking without leaving the app
- **Train & Flight Support** — Expand beyond buses to cover trains and flights
- **Database Backend** — Replace localStorage with PostgreSQL for persistent user data
- **PWA / Mobile App** — Offline support and push notifications via Progressive Web App
- **Multi-language** — Support Tamil, Hindi, Kannada for regional users
- **Price Prediction** — ML model to predict fare changes using historical data
- **Real-time Seat Map** — Let users pick seats inside the app before booking
- **Fine-tuned NLP Model** — Train a domain-specific model for richer intent understanding

8. Summary

We built a complete, working AI-powered bus ticketing assistant using **React + TypeScript** on the frontend, **FastAPI + Python** on the backend, **OpenRouter LLM + rule-based parser** for AI, and **Playwright** for live data scraping. The system handles the entire journey from a plain-English user query to filtered, live bus results with booking links — in one seamless interface.

The project demonstrates how modern AI, web scraping, and thoughtful UI design can be combined to solve a real, everyday problem for millions of bus travellers in India.